

Air Quality Data Analysis

Delhi Air-Quality Observation

Project Overview

Analysis of Delhi air-quality observation using Python

Tools: Pandas, NumPy, Matplotlib and Seaborn.

Focus: PM2.5, PM10 and other air-quality parameters.

Air Quality Data Analysis Report

- Python
- Pandas
- Matplotlib
- Seaborn

AIR QUALITY DATA ANALYSIS

This project analyzes Delhi air-quality observations using Python, Pandas, Matplotlib, and Seaborn. The notebook loads a CSV dataset, checks its structure, handles missing/duplicate records, and creates visualizations for PM2.5 and other pollutants.

TOOLS AND LIBRARIES

The project uses Pandas for data handling, NumPy for numerical operations, Matplotlib for plotting, and Seaborn for statistical visualization.

PROJECT WORKFLOW

- Load the Delhi air-quality CSV file.
- Inspect rows, columns, shape, data types, and summary statistics.
- Check missing values and remove duplicates / null records.
- Convert the date field into datetime format and create a Month column.
- Analyze PM2.5 using line, bar, histogram, box, heatmap, and scatter plots.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

DATASET LOADING

FILE UPLOAD

The notebook uses Google Colab's file-upload widget and then reads the uploaded delhiaqi.csv file with Pandas.

Code

```
from google.colab import files
uploaded = files.upload()
```

READ CSV FILE

Code

```
import pandas as pd
df = pd.read_csv("delhiaqi.csv")
df.head()
```

Explanation

The loaded dataset contains a date field and air-quality measurements including CO, NO, NO2, O3, SO2, PM2.5, PM10, and NH3.

INITIAL DATA VIEW

HEAD()

Code

```
df.head()
```

Output

date	co	no	no2	o3	so2	pm2_5	pm10	nh3
2023-01-01 00:00	1655.58	1.66	39.41	5.90	17.88	169.29	194.64	5.83
2023-01-01 01:00	1869.20	6.82	42.16	1.99	22.17	182.84	211.08	7.66
2023-01-01 02:00	2510.07	27.72	43.87	0.02	30.04	220.25	260.68	11.40
2023-01-01 03:00	3150.94	55.43	44.55	0.85	35.76	252.90	304.12	13.55
2023-01-01 04:00	3471.37	68.84	45.24	5.45	39.10	266.36	322.80	14.19

Explanation

The first five observations show hourly measurements beginning on 1 January 2023. PM2.5 and PM10 are included as major particulate-matter variables.

END OF DATASET

TAIL()

Code

```
df.tail()
```

Output

date	co	no	no2	o3	so2	pm2_5	pm10	nh3
2023-01-24 04:00	1762.39	4.64	37.01	33.26	30.52	231.15	289.84	6.27
2023-01-24 05:00	1735.69	6.82	34.96	46.49	34.33	225.08	280.52	9.12
2023-01-24 06:00	1922.61	8.16	40.10	56.51	43.39	242.49	296.07	12.54
2023-01-24 07:00	1361.85	9.05	52.78	71.53	100.14	165.67	191.82	7.47
2023-01-24 08:00	1134.87	8.61	56.89	80.11	110.63	123.76	140.26	5.51

Explanation

The final displayed records are from 24 January 2023. This confirms that the dataset covers a sequence of hourly observations.

DATASET STRUCTURE

SHAPE AND COLUMNS

Code

```
df.shape  
  
df.columns
```

Output

Property	Result
Rows	561
Columns	9
Date column	date
Pollutant columns	co, no, no2, o3, so2, pm2_5, pm10, nh3

Explanation

The notebook reports a shape of (561, 9). Therefore, the working dataframe has 561 observations and 9 columns.

DATA TYPES AND SUMMARY

DATA INFORMATION

Code

```
df.info()
```

The dataset contains one date column and eight numerical air-quality columns. The project then uses `describe()` to examine count, mean, standard deviation, minimum, maximum, and quartiles.

DESCRIPTIVE STATISTICS

Code

```
df.describe()
```

Output

Variable	Min	Max	Std. Dev.
co	654.22	16876.22	3227.74
no	0.00	425.58	83.90
no2	13.37	263.21	42.47

o3	0.00	164.51	39.98
so2	5.25	511.17	61.07
pm2_5	60.10	1310.20	227.36
pm10	69.08	1499.27	271.29
nh3	0.63	267.51	36.56

DATA QUALITY CHECK

MISSING VALUES

Code

```
df.isnull().sum()
```

Output

Column	Missing values
date	0
co	0
no	0
no2	0
o3	0
so2	0
pm2_5	0
pm10	0
nh3	0

Explanation

The notebook output shows zero missing values in every column. The project still includes dropna() as a defensive cleaning step.

DUPLICATE AND NULL CLEANING

Code

```
df.drop_duplicates(inplace=True)
```

```
df.dropna(inplace=True)
```

WHY THIS STEP IS IMPORTANT

- Enables time-based plotting.
- Makes monthly aggregation possible.
- Allows the PM2.5 trend to be studied in chronological order.
- Keeps the analysis organized around the original observation time.

PM2.5 TREND OVER TIME

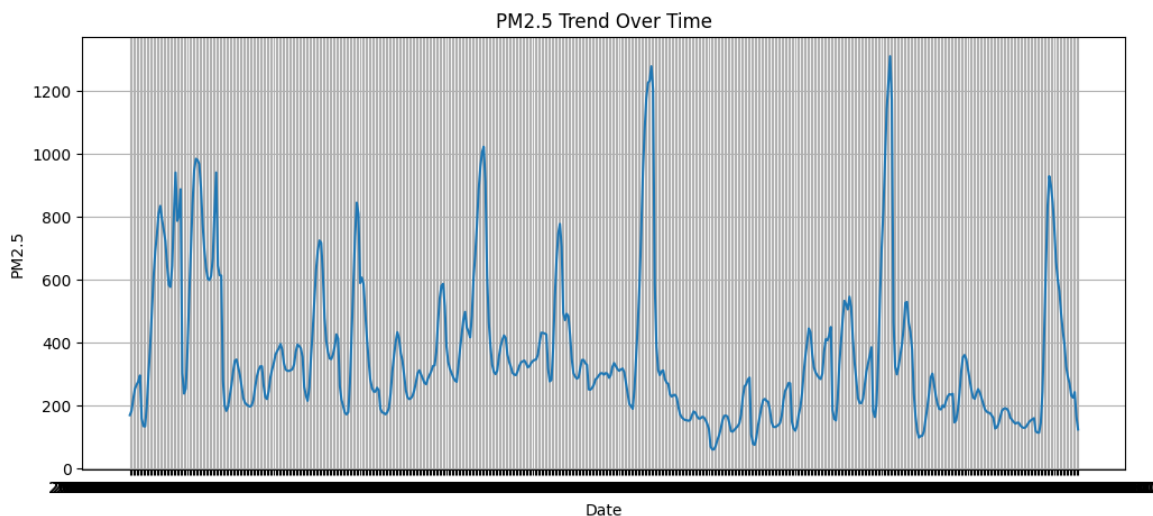
LINE CHART

Code

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))
plt.plot(df["date"], df["pm2_5"])
plt.title("PM2.5 Trend Over Time")
plt.xlabel("Date")
plt.ylabel("PM2.5")
plt.grid(True)
plt.show()
```

Output



Explanation

The line chart displays how PM2.5 changes across the recorded dates. Peaks and falls can be identified directly from the time-series pattern.

AVERAGE PM2.5 BY MONTH

BAR GRAPH

Code

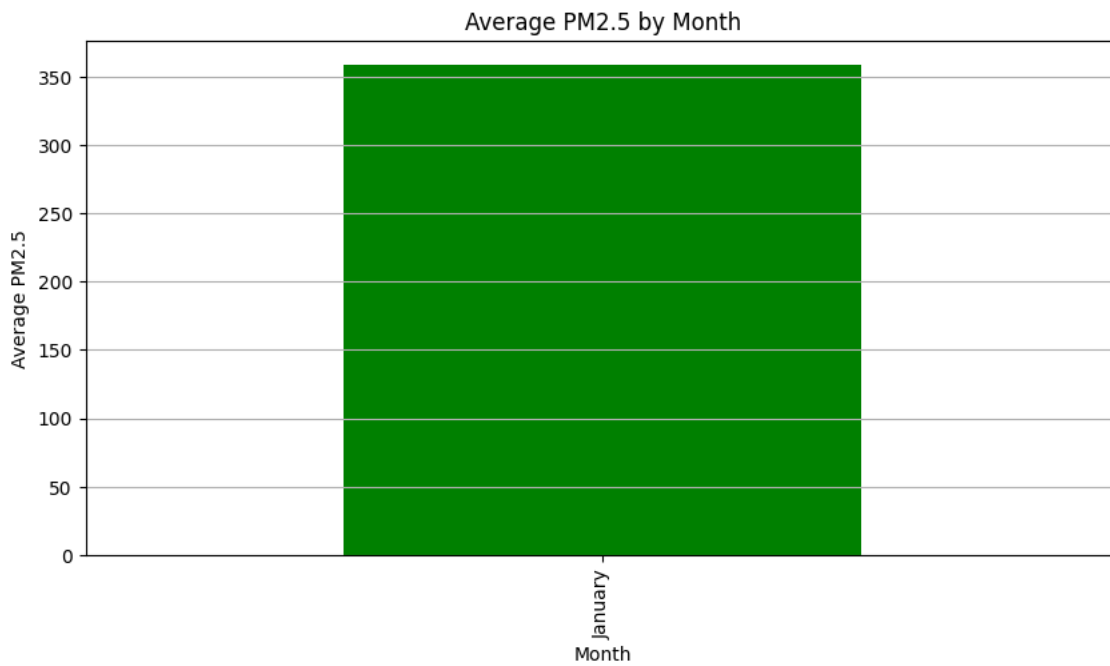
```
import pandas as pd
import matplotlib.pyplot as plt

df["date"] = pd.to_datetime(df["date"])
df["Month"] = df["date"].dt.month_name()

plt.figure(figsize=(10,5))
df.groupby("Month")["pm2_5"].mean().plot(kind="bar", color="green")

plt.title("Average PM2.5 by Month")
plt.xlabel("Month")
plt.ylabel("Average PM2.5")
plt.grid(axis="y")
plt.show()
```

Output



Explanation

The bar chart compares the average PM2.5 value for each month represented in the data. Because the dataset spans January 2023, the monthly grouping is based on the available observations.

DISTRIBUTION OF PM2.5

HISTOGRAM

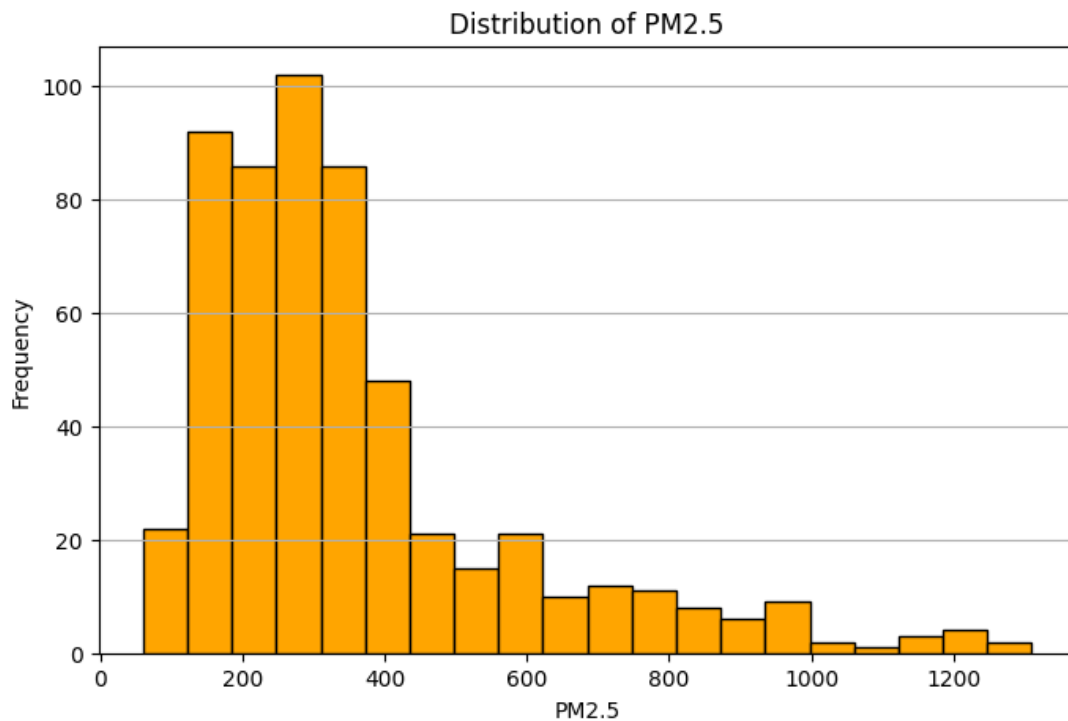
Code

```
plt.figure(figsize=(8,5))

plt.hist(df["pm2_5"], bins=20, color="orange", edgecolor="black")

plt.title("Distribution of PM2.5")
plt.xlabel("PM2.5")
plt.ylabel("Frequency")
plt.grid(axis="y")
plt.show()
```

Output



Explanation

The histogram divides PM2.5 values into 20 bins and shows how frequently values occur in each interval. It helps identify the concentration and spread of particulate-matter observations.

PM2.5 VARIABILITY

BOX PLOT

Code

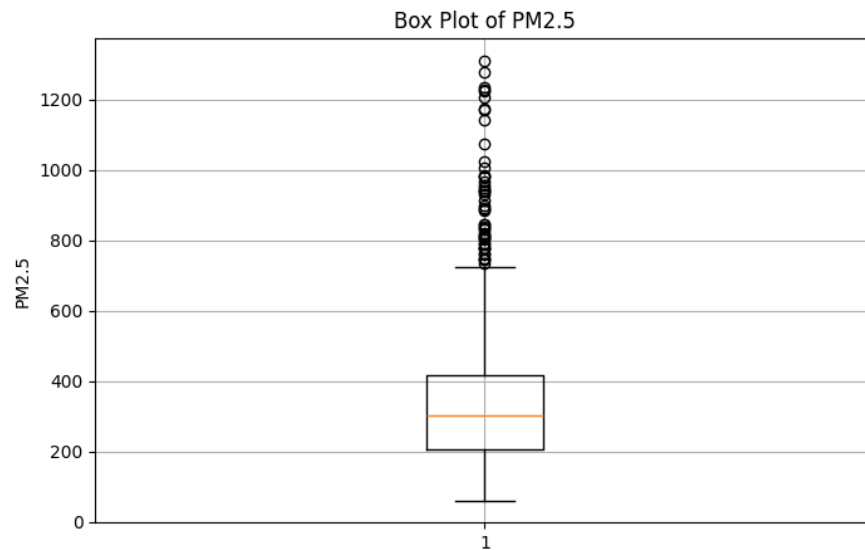
```
plt.figure(figsize=(8,5))

plt.boxplot(df["pm2_5"])

plt.title("Box Plot of PM2.5")
plt.ylabel("PM2.5")
plt.grid(True)

plt.show()
```

Output



Explanation

The box plot summarizes the distribution of PM2.5 using quartiles. It is useful for seeing the central spread and identifying values that lie unusually far from the typical range.

MONTHLY PM2.5 TREND

LINE PLOT BY MONTH

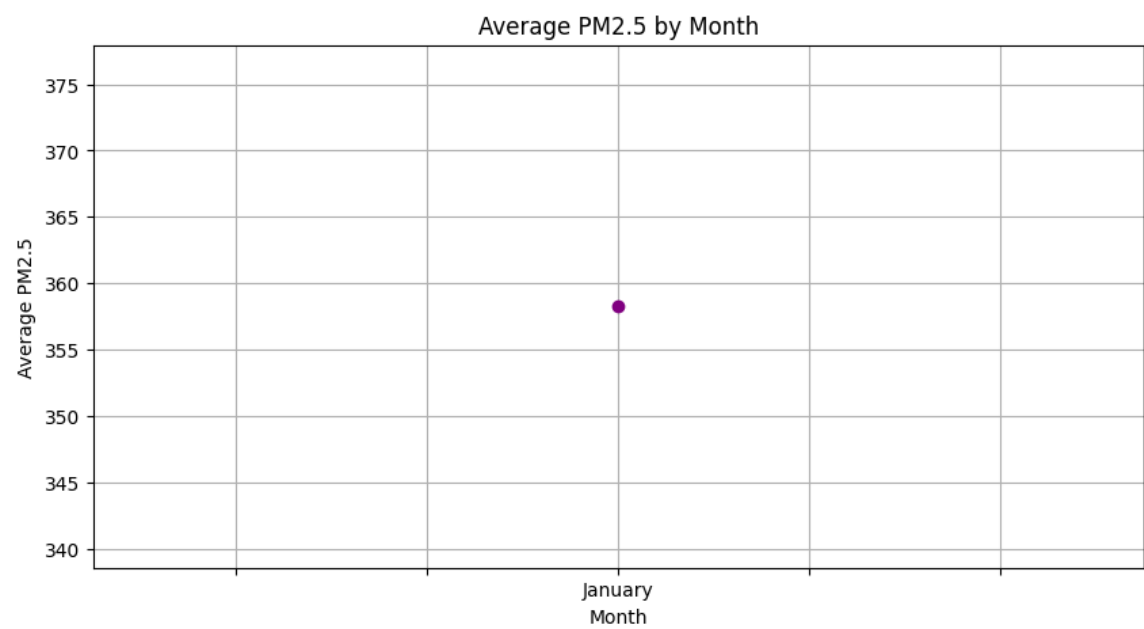
Code

```
plt.figure(figsize=(10,5))

df.groupby("Month")["pm2_5"].mean().plot(
    kind="line",
    marker="o",
    color="purple"
)

plt.title("Average PM2.5 by Month")
plt.xlabel("Month")
plt.ylabel("Average PM2.5")
plt.grid(True)
plt.show()
```

Output



Explanation

This visualization presents the same monthly aggregation as a line chart, making the direction of change between consecutive month groups easier to follow.

AIR QUALITY CORRELATION

SEABORN HEATMAP

Code

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))

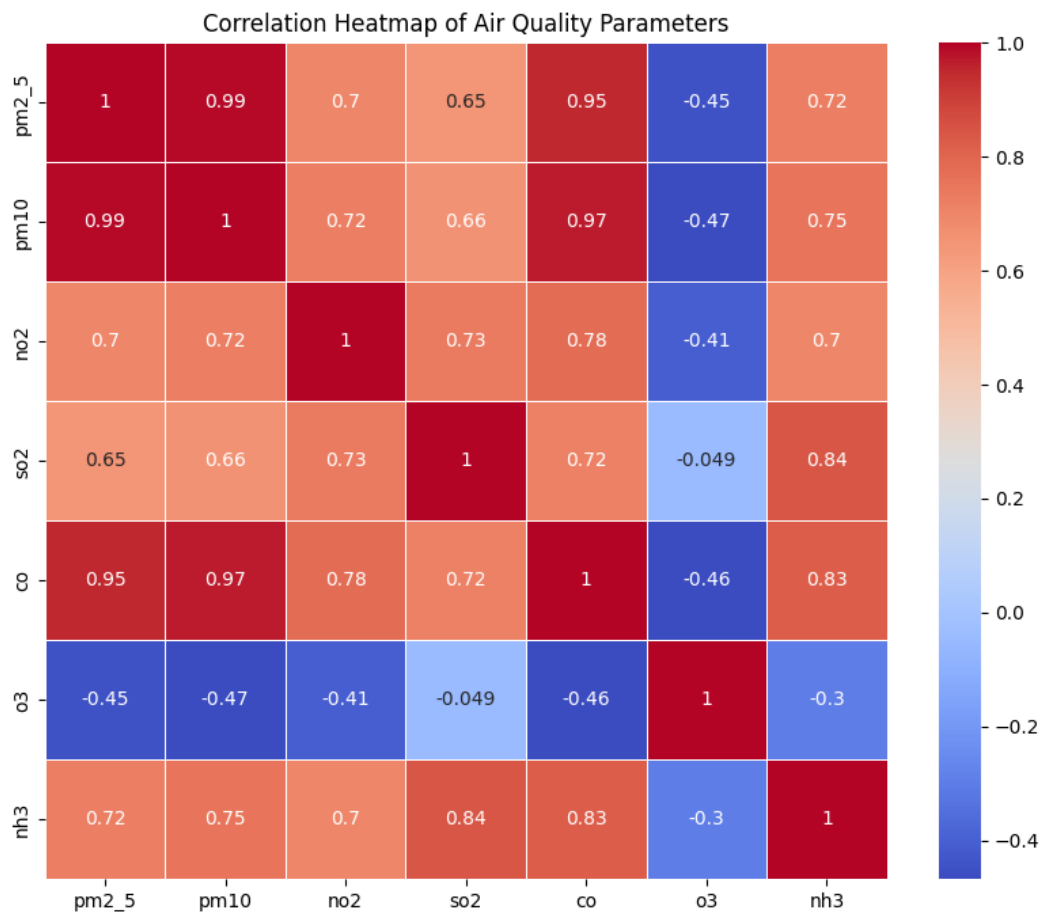
numerical_cols = ['pm2_5', 'pm10', 'no2', 'so2', 'co', 'o3', 'nh3']

correlation_matrix = df[numerical_cols].corr()

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    linewidths=0.5
)

plt.title("Correlation Heatmap of Air Quality Parameters")
plt.show()
```

Output



Explanation

The heatmap represents pairwise correlations between numerical air-quality variables. The `annot=True` setting places the correlation values inside the cells.

PM2.5 VS PM10

SEABORN SCATTER PLOT

Code

```
import seaborn as sns
import matplotlib.pyplot as plt

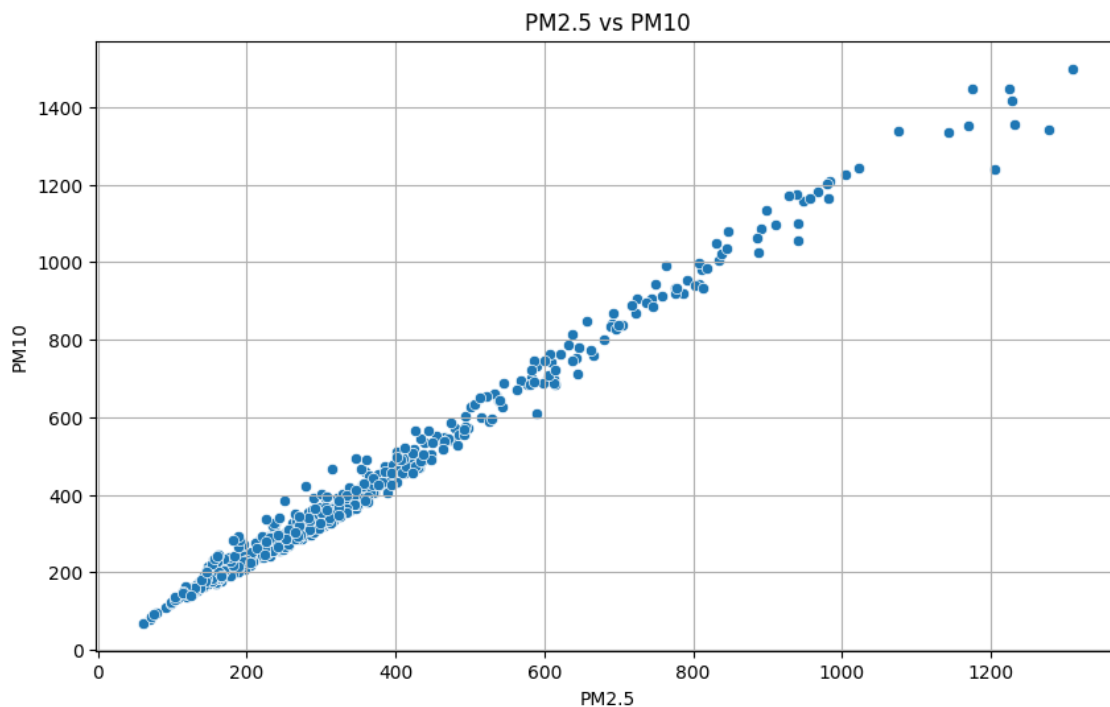
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="pm2_5",
    y="pm10"
)

plt.title("PM2.5 vs PM10")
plt.xlabel("PM2.5")
plt.ylabel("PM10")
plt.grid(True)

plt.show()
```

Output



Explanation

Each point represents an observation. The scatter plot is used to visually inspect the relationship between PM2.5 and PM10 values.

PROJECT SUMMARY

KEY LEARNINGS

This project demonstrates a complete beginner-to-intermediate data-analysis workflow on an air-quality dataset. The notebook loads the data, inspects its structure, checks data quality, performs basic cleaning, prepares dates, and applies multiple visualization techniques.

VISUALIZATIONS USED

- PM2.5 trend over time — line chart
- Average PM2.5 by month — bar chart
- PM2.5 distribution — histogram
- PM2.5 variability — box plot
- Average PM2.5 by month — line plot
- Air-quality parameter relationships — correlation heatmap
- PM2.5 vs PM10 — scatter plot

CONCLUSION

The analysis provides a visual understanding of particulate pollution and its relationship with other measured air-quality parameters. Matplotlib provides direct control over charts, while Seaborn makes statistical visualizations such as heatmaps and scatter plots convenient.

Project source: `Intermediate_project(2).ipynb`